

Technical Design

Design summary

Use a single web application backed by PostgreSQL. Route handlers call a small booking service; the service owns transactions and state transitions; repositories contain database queries. The database is the final authority for uniqueness and capacity.

The key operation is `applyPaymentResult`. For a successful payment result it starts a transaction, locks the booking and then the `trial_classes` row, records an idempotent payment attempt, re-checks duplicate and confirmed occupancy, and then updates the booking. Serializing confirmation per class makes the last-seat outcome deterministic and easy to explain.

Component boundaries

- Minimal UI: sends commands and renders returned state; never decides final availability.
- Route handlers: parse input, apply demo identity, map domain errors to HTTP responses.
- Booking service: owns workflow, transactions, and status transitions.
- Repositories: execute explicit database operations.
- PostgreSQL: enforces unique/idempotency constraints and serializes class confirmation.
- Mock payment adapter: produces success or failure without a real provider.

Pattern selection

The take-home uses a local transaction and an explicit state machine, not a distributed Saga. All implemented booking invariants are decided inside one PostgreSQL transactional boundary, while payment is a recorded mock result. Calling this a Saga would overstate the architecture.

The production workflow is Saga-like once real payment operations commit independently: authorize payment, claim the seat, capture payment, and compensate with a void or refund when a later step fails. Every step must be idempotent, and a reconciliation worker must recover incomplete workflows.

Kafka is deliberately excluded. The current requirements do not demonstrate high event volume, durable replay, or multiple independently deployed consumers. If reliable event publication becomes necessary, first write a transactional outbox record in the same transaction as booking confirmation. A background publisher can deliver it to a broker. Kafka should be selected only when consumer count, replay, ordering, throughput, and operational ownership justify it.

Spec-driven delivery

Before implementation, freeze a compact baseline consisting of:

- product requirements and acceptance criteria;
- technical plan and architecture decisions;
- data model and API contracts;
- dependency-ordered tasks;

- a verification matrix mapping acceptance criteria to tests.

The specification guides implementation but does not replace verification. If behavior changes, update the relevant requirement, decision, task, and test together.

Git workflow

Use `main` plus one short-lived `feature/trial-booking` branch and small, reviewable commits. Merge through one pull request after verification. Full GitFlow is not recommended because the take-home has no long-running `develop` integration branch, release train, or hotfix lifecycle.

Data model

parents

Column	Type	Notes
<code>id</code>	uuid	Primary key
<code>name</code>	text	Synthetic display name
<code>email</code>	text	Synthetic, unique

students

Column	Type	Notes
<code>id</code>	uuid	Primary key
<code>parent_id</code>	uuid	FK to parents
<code>name</code>	text	Synthetic display name

trial_classes

Column	Type	Notes
<code>id</code>	uuid	Primary key
<code>title</code>	text	Subject/level label
<code>starts_at</code>	timestampz	UTC
<code>capacity</code>	smallint	Check <code>capacity > 0</code> ; seed with 4

bookings

Column	Type	Notes
<code>id</code>	uuid	Primary key

Column	Type	Notes
student_id	uuid	FK to students
trial_class_id	uuid	FK to trial classes
status	enum/text	Controlled status vocabulary
request_key	text	Unique idempotency key for creation
created_at	timestampz	UTC
updated_at	timestampz	UTC
confirmed_at	timestampz nullable	Set only on confirmation

Recommended constraint:

```
CREATE UNIQUE INDEX one_active_booking_per_student_class
ON bookings (student_id, trial_class_id)
WHERE status IN ('pending_payment', 'confirmed');
```

This prevents two live checkout paths for the same child/class while allowing retry after a terminal failure.

payment_attempts

Column	Type	Notes
id	uuid	Primary key
booking_id	uuid	FK to bookings
idempotency_key	text	Unique
provider_reference	text nullable	Unique when present
result	enum/text	succeeded or failed for demo
amount_minor	integer	Optional but useful audit data
created_at	timestampz	UTC

API surface

Method and route	Purpose	Key responses
GET /api/students	List children for seeded parent	200
GET /api/trial-classes	List classes and indicative availability	200
POST /api/bookings	Create/reuse pending booking	201, 200 replay, 409 active duplicate

Method and route	Purpose	Key responses
POST /api/bookings/:id/payment-result	Record mock result and transition booking	200, 409 invalid/terminal transition
GET /api/bookings/:id	Fetch status	200, 404
GET /api/trial-classes/:id/roster	Return confirmed students only	200, 404

Every command endpoint accepts an `Idempotency-Key`. Responses use stable machine-readable error codes such as `ACTIVE_BOOKING_EXISTS`, `CLASS_FULL`, `INVALID_TRANSITION`, and `IDEMPOTENCY_KEY_REUSED`.

Atomic confirmation algorithm

Conceptual transaction for a successful payment result:

```
BEGIN
  load booking FOR UPDATE
  if payment idempotency key already exists: return its recorded outcome
  reject invalid terminal-state transitions
  lock parent trial_class row FOR UPDATE
  record successful payment attempt
  count confirmed bookings for this class
  if count >= capacity:
    set booking = capacity_unavailable
  else:
    set booking = confirmed, confirmed_at = now
COMMIT
```

Lock order is always booking first, then class, for every code path that needs both. For a high-throughput production system, lock the class first consistently or use a dedicated inventory row; consistency matters more than which order is chosen. The implementation should document and test its actual order.

Why the class row lock works:

1. Both final-seat confirmations target the same class row.
2. Only one transaction obtains the row lock first.
3. The winner sees three confirmed bookings and becomes the fourth.
4. The loser waits, then sees four confirmed bookings and becomes `capacity_unavailable`.
5. No transaction holds the database lock while calling an external service.

Duplicate protection

Defense in depth:

1. UI disables submission after the first click for user experience only.
2. API requires an idempotency key and returns the original result on replay.

3. Service checks for an existing active booking.
4. A partial unique database index prevents concurrent creation of multiple active bookings.

Payment behavior

- A failed result is recorded and transitions `pending_payment -> payment_failed`.
- A successful result triggers the atomic capacity check.
- A successful result with no seat transitions to `capacity_unavailable`; it never joins the roster.
- Conflicting later results for the same booking are rejected and logged.
- In production, prefer authorize-then-capture, with a void/refund path for capacity loss.

Availability and roster queries

Availability is a snapshot:

```
available_seats = max(capacity - confirmed_count, 0)
```

It is suitable for display but cannot authorize confirmation. Roster queries join bookings to students and include `bookings.status = 'confirmed'` in the database query.

Transaction and isolation notes

PostgreSQL `READ COMMITTED` is sufficient with an explicit row lock on the class because all capacity-changing operations follow the same lock protocol. `SERIALIZABLE` is a valid alternative, but it requires retry handling and is less direct to explain in a short exercise. A transaction alone, without a lock or serializable predicate protection, is not enough.

Error and recovery policy

- Validation error: `400` with field details.
- Missing resource: `404`.
- Duplicate/invalid transition/capacity conflict: `409` with domain code.
- Idempotent replay: return the previously stored result, normally `200`.
- Unexpected database failure: rollback and return `500`; do not expose internal details.
- Every response includes a request ID for correlation.

Security notes

For the demo, use a seeded identity. In production:

- Verify the parent owns the student.
- Require staff role for roster access.
- Never trust amount, payment success, parent ID, or booking status supplied by the browser.
- Authenticate provider callbacks and prevent replay.
- Avoid logging child PII or payment secrets.

Observability after release

Track:

- booking attempts by outcome;
- payment failures and capacity losses after payment success;
- duplicate/idempotent replay counts;
- confirmed occupancy above capacity (should always be zero);
- confirmation latency and lock wait time;
- bookings stuck in `pending_payment` ;
- refund/void-required events.

Alert immediately if any class exceeds capacity or a non-confirmed booking appears in roster output.

Alternatives considered

- In-memory store: fastest setup, but does not demonstrate multi-process correctness.
- SQLite: convenient, but its coarse write locking and environment-specific behavior make the final-seat explanation less representative.
- Seat holds: good production UX, but expiry, cleanup, and payment coordination are too much for this timebox.
- Optimistic capacity counter: viable with a conditional update, but a class-row lock plus count is easier to inspect and test here.
- Distributed Saga: appropriate after adding real independently committed payment operations, but unnecessary for the mocked single-database slice.
- Kafka: appropriate for justified replay, throughput, and multi-consumer requirements, but operationally disproportionate here.
- Full GitFlow: useful for some scheduled-release teams, but unnecessary for one short-lived take-home branch.

Production evolution

1. Replace mock payment success with authorization.
2. Claim the seat in the existing atomic database transaction.
3. Write an outbox event in that same transaction.
4. Capture payment after the seat is secured.
5. Void the authorization when capacity is unavailable.
6. Refund or flag reconciliation if capture or void outcomes are uncertain.
7. Publish outbox events idempotently for notifications, analytics, and audit consumers.
8. Introduce Kafka only if the measured event and consumer requirements justify operating it.